**Hitesh Rathore (HR)** Notes
linkedin.com/in/hitesh-rathore-835279385

# STRING PRIMITIVE Vs STRING OBJECT
## WRAPPER OBJECTS & AUTO BOXING
(FIRST PRINCIPLES APPROACH)

Until now we know:
Strings are Primitive Values.

**QUESTION** If Strings are primitive values, then why does this work?

EXAMPLE
```
let name = "Hitesh";
name.length
name.toUpperCase()
name.includes("Hi")
```

**QUESTION** Primitive values should just store data.

THEN HOW DO THEY HAVE
- Properties?
- Methods?

THIS IS ONE OF THE MOST IMPORTANT JAVASCRIPT INTERNALS TOPICS

### PART 1 — THE CONFUSION

Question: What is: `"Hitesh"` ? → Answer: A Primitive String ✓

Question: Can primitive values contain methods? → Answer: NO ✗

Why? Primitive values are not objects.

Question: Then how does this work? → Example: `"Hitesh".toUpperCase()`

Question: Where did toUpperCase() come from? → Answer: Wrapper Objects

### STRING PRIMITIVE

| | | |
|---|---|---|
| Question: How do we create a normal String? | Example: | `let name = "Hitesh";` |
| Type: `typeof name` | Output: | `"string"` |

Important: This is a Primitive String.

Visualization: name → "Hitesh"

No object exists

### STRING OBJECT

JavaScript also provides: String Objects — Example: `let name = new String("Hitesh");`

Question: What is its type? — Example: `typeof name` — Output: `"object"`

Important: This is NOT a Primitive String.

Question: `typeof "Hitesh"` — Output: `"string"`

This is an object: name → String Object → { internalValue: "Hitesh" }

Question: `typeof new String("Hitesh")` — Output: `"object"`

VERY IMPORTANT — Primitive String `"Hitesh"` ≠ String Object `new String("Hitesh")`

## WHY STRING OBJECTS EXIST

**QUESTION** Why did JavaScript create String Objects?

**ANSWER** To provide:
- ✓ Methods
- ✓ Properties
- ✓ Functionality

**EXAMPLES** `toUpperCase()` | `toLowerCase()` | `slice()` | `includes()` | `replace()`

**QUESTION** But we never write: `new String()` Then how do methods work?

**AUTO BOXING** This is the secret.

**QUESTION** What happens here? `"Hitesh".toUpperCase()`

DOES THE PRIMITIVE STRING ACTUALLY HAVE THIS METHOD? NO

THEN WHAT HAPPENS? JavaScript temporarily creates a String Object.

INTERNAL IDEA: "Hitesh" → new String("Hitesh") → toUpperCase() → Result Returned → Temporary Object Destroyed

THIS PROCESS IS CALLED **AUTO BOXING**

DEFINITION: Auto Boxing means: JavaScript temporarily wraps a primitive value inside an object so that methods and properties can be used.

VISUALIZATION: Primitive String → Temporary String Object → Method Execution → Object Removed

## STEP BY STEP INTERNAL WORKING

CONSIDER:
```
let name = "Hitesh";
name.toUpperCase();
```

1. Primitive String exists. `"Hitesh"`
2. JavaScript sees: `toUpperCase()`
3. Primitive String has no methods. What now?
4. JavaScript automatically creates: `new String("Hitesh")`
5. Method executes on that object.
6. Result returned.
7. Temporary object destroyed.

OUTPUT: "HITESH"

## WHY .length WORKS

**QUESTION** How does this work? `"Hitesh".length`

INTERNAL IDEA
- "Hitesh"
- ↓ Temporary String Object
- ↓ Read length Property
- ↓ Return 6
- ↓ Destroy Object

## VERY IMPORTANT INTERVIEW QUESTION

```
let name = "Hitesh";
name.age = 19;
console.log(name.age);
```

**QUESTION** Output?

**ANSWER** undefined

**WHY?** Let's see.

LINE 1 — `let name = "Hitesh";`
Memory: name → "Hitesh" (Primitive String)
A primitive string is stored in memory.

LINE 2 — `name.age = 19;`
A String Primitive cannot store custom properties.
So JavaScript temporarily creates a String Object:
"Hitesh" ↓ `new String("Hitesh")`

Temporary Object:
```
0 : "H"
1 : "i"
2 : "t"
3 : "e"
4 : "s"
5 : "h"
-----------
length : 6
age :   19
```

The property is successfully added: `age = 19`
However, this object is only temporary.
As soon as the statement finishes executing: `new String("Hitesh")` ✗ Destroyed

LINE 3 — `console.log(name.age);`
Now JavaScript creates another temporary String Object:
"Hitesh" ↓ `new String("Hitesh")`

This is a completely new object:
```
0 : "H"
1 : "i"
2 : "t"
3 : "e"
4 : "s"
5 : "h"
-----------
length : 6
```

Notice carefully: `age` does not exist in this object.
The age property existed only on the previous temporary object, and that object has already been destroyed.

FINAL RESULT
`name.age` → undefined
`console.log(name.age)` ↓ undefined

KEY IDEA
- ✓ Primitive strings do not permanently store custom properties.
- ✓ When a property or method is accessed, JavaScript creates a temporary String Object.
- ✓ Any custom property added to that temporary object disappears when the object is destroyed.
- ✓ That is why `name.age` returns undefined.

## STRING OBJECT COMPARISON

| **QUESTION** What happens here? | **QUESTION** What happens here? |
|---|---|
| EXAMPLE `"Hitesh" === "Hitesh"` | EXAMPLE `new String("Hitesh") === new String("Hitesh")` |
| OUTPUT `true` | OUTPUT `false` |
| WHY? Primitive Values compared by value. | WHY? Two different objects. |

VISUALIZATION
Object A: String Object ↓ "Hitesh"
Object B: String Object ↓ "Hitesh"
✗ Different References ✗

DIFFERENT REFERENCES `false`

## SHOULD WE USE new String() ?

QUESTION: Should developers create String Objects?

ANSWER: Almost Never.

RECOMMENDED: `let name = "Hitesh";`

AVOID: `let name = new String("Hitesh");`

WHY?
- Creates unnecessary objects.
- Can create comparison confusion.
- Slower than primitives.

## OTHER WRAPPER OBJECTS

JavaScript provides wrapper objects for:

| STRING | NUMBER | BOOLEAN |
|---|---|---|
| Tt String | 123 Number | true Boolean |

EXAMPLES: `new String("Hello")` | `new Number(10)` | `new Boolean(true)`

ALL RETURN Objects

## new Boolean() TRAP

QUESTION
```
if (new Boolean(false)) {
  console.log("Executed");
}
```

What many people think: `false`
So they expect: Not Executed
Reality: Executed

WHY?

STEP 1 — `new Boolean(false)` does NOT create a Boolean Primitive. It creates a Boolean Object.
Visualization: `new Boolean(false)` ↓ Boolean Object

STEP 2 — The `if` statement does NOT check the internal boolean value. It checks whether the value is Truthy or Falsy. `if (value)`

STEP 3 — All Objects are Truthy.
Example:
- `if ({})` → ✓ Executes
- `if ([])` → ✓ Executes
- `if (new Boolean(false))` → ✓ Executes

TRAP
`Boolean(false)` returns: `false`
But
`new Boolean(false)` returns: Boolean Object
These are completely different things.

COMPARISON
`Boolean(false)` ↓ `false` ↓ Boolean Primitive
`new Boolean(false)` ↓ Boolean Object ↓ Object ↓ Truthy

INTERVIEW RULE
`Boolean(false)` returns a Primitive.
`new Boolean(false)` returns an Object.
All Objects are Truthy.
Therefore: `if (new Boolean(false))` will execute.
Output: Executed

KEY TAKEAWAY
- ✓ Avoid using `new Boolean()`, `new Number()`, `new String()`.
- ✓ They create objects, not primitives.
- ✓ All objects are truthy, which can lead to unexpected behavior.

## DANGEROUS TRAPS

**TRAP 1**

`if (new Boolean(false))` ↓ Output: `true branch`
Because object is truthy.

`new Number(10) === 10`
Output: `false`
Reason: Object !== Primitive
`===` (Strict Equality) does NOT perform type conversion.
Before checking values, it checks types.
Left → Object | Right → Number Primitive
Types are different.
Therefore: `false`

**TRAP 2**

`new String("Hi") === "Hi"` Output: `false` Reason: Object !== Primitive

**TRAP 3**

```
console.log(new Number(10) == 10);   ??
```

| | |
|---|---|
| STEP 1 | Left side: `new Number(10)` creates a Number Object. |
| STEP 2 | Right side: `10` is a Number Primitive. |
| STEP 3 | `==` sees: Object == Primitive<br>When one side is an Object and the other side is a Primitive, JavaScript tries to convert the Object into a Primitive.<br>So internally: `new Number(10).valueOf()`<br>returns: `10` |
| STEP 4 | Now comparison becomes: `10 == 10`<br>Result: `true` |

## MENTAL MODEL

- Primitive String
- ↓ No Methods
- ↓ JavaScript Creates Temporary String Object
- ↓ Methods Become Available
- ↓ Method Executes
- ↓ Temporary Object Destroyed

- "Hitesh"
- ↓ Auto Boxing
- ↓ String Object
- ↓ Method/Property Access
- ↓ Result Returned

---

# MEMORY MODEL
(FIRST PRINCIPLES APPROACH)

Until now we know:
- ✓ Strings are Primitive Values
- ✓ Strings are Immutable
- ✓ JavaScript uses UTF-16 based Strings
- ✓ Wrapper Objects & Auto Boxing

| QUESTION | QUESTION | QUESTION | QUESTION |
|---|---|---|---|
| How are Strings actually stored? | Why are Strings immutable? | What happens when we modify a String? | Does JavaScript change the existing String? Or Create a new String? |

## PART 1 HOW STRINGS ARE STORED?

CONSIDER: EXAMPLE `let name = "Hitesh";`

CONCEPTUAL MEMORY MODEL: name ↓ "Hitesh"

QUESTION What does the variable contain? ANSWER The String Value

QUESTION How is "Hitesh" stored internally? ANSWER As a sequence of UTF-16 Code Units.

THEREFORE Variable stores the String value directly.
VISUALIZATION name / "Hitesh"

VISUALIZATION
```
"H"  →  UTF-16 Unit
"i"  →  UTF-16 Unit
"t"  →  UTF-16 Unit
"e"  →  UTF-16 Unit
"s"  →  UTF-16 Unit
"h"  →  UTF-16 Unit
```

TOTAL 6 UTF-16 Code Units

IMPORTANT JavaScript does NOT store: "Hitesh" as one giant character block.

IT STORES A sequence of UTF-16 Code Units.

## PART 2 WHY ARE STRINGS IMMUTABLE?

QUESTION Why didn't JavaScript allow: → EXAMPLE
```
let str = "Hello";
str[0] = "J";
```
→ EXPECTED "Jello" → ACTUAL "Hello"

QUESTION Why? — ANSWER Strings are Immutable. — QUESTION But WHY are they immutable? — REASON Performance Optimization

QUESTION What if thousands of variables contain: "Hello" ?

EXAMPLE
```
let a = "Hello";
let b = "Hello";
let c = "Hello";
```

MEMORY SHARING: a, b, c → "Hello" Safe

QUESTION Can JavaScript reuse memory? ANSWER YES
WHY? Because Strings cannot change.

IF STRINGS WERE MUTABLE Shared memory would become dangerous.

IMMUTABILITY MAKES Memory Sharing Safe.

INTERVIEW QUESTION
```
let str = "hello";
str.toUpperCase();
console.log(str);
```
→ OUTPUT `"hello"` — WHY? Result was not stored.

`toUpperCase()` Creates a NEW String » BUT Does not modify the old one.

## PART 3 STRING CONCATENATION

QUESTION What happens during:

EXAMPLE
```
let str = "Hello";
str += " World";
```

QUESTION Does JavaScript modify: "Hello"

ANSWER NO

IMPORTANT Original String never changes.

INTERNAL IDEA
- Step 1 Existing String ↓ "Hello"
- » Step 2 New String ↓ "Hello World" is created.
- » Step 3 Variable updated.

VISUALIZATION
Before: str ↓ "Hello"
After: str ↓ "Hello World"

NEW STRING ALWAYS CREATED

EXAMPLE
```
let a = "Hi";
let b = a;
a += "!";
```
QUESTION What is b ? » ANSWER "Hi" » WHY? a received a NEW String. b still has old value.

VISUALIZATION
Initially: a, b → "Hi" — Both a and b point to the same String "Hi".
» After: a → "Hi!", b → "Hi" — a now points to NEW String "Hi!". b still points to old String "Hi".

## PART 4 METHODS AND IMMUTABILITY

QUESTION What do String methods usually do? → ANSWER Create New Strings.

EXAMPLES
- `toUpperCase()`
- `toLowerCase()`
- `slice()`
- `substring()`
- `replace()`
- `trim()`

QUESTION Do they modify the original String? → ANSWER NO

EXAMPLE
```
let str = " hello ";
str.trim();
console.log(str);
```

OUTPUT `" hello "`

WHY? `trim()` returns a NEW String. → CORRECT WAY `str = str.trim();`

## PART 5 INTERVIEW EDGE CASE

QUESTION Can we add properties to a String? → EXAMPLE
```
let str = "Hello";
str.age = 10;
```
QUESTION Will age be stored? → ANSWER NO

WHY? Primitive Values are immutable. → Temporary wrapper object is destroyed. → OUTPUT `undefined`

## PART 6 STRING REUSE CONCEPT

CONSIDER: EXAMPLE
```
let a = "JavaScript";
let b = "JavaScript";
```

QUESTION: Can JavaScript internally reuse memory? ANSWER Often YES.

WHY? Because Strings are immutable.

IMPORTANT: This optimization is often called: String Interning
"JavaScript" ← a, b (Shared Memory)

NOTE: We will study String Interning in a later advanced phase.

## MENTAL MODEL

Strings are Immutable → Methods Never Modify Original String → Methods Create New Strings → Variables Can Be Reassigned → But Original Strings Never Change

"Hello" → toUpperCase() → "HELLO" (New String) | "Hello" remains unchanged. ✓